

Rapport d'Apprentissage : OverTheWire - Bandit

(Niveaux 0 à 12 – Manipulation des données)

💡 Qu'est-ce que Bandit ?:

Bandit est une succession de **challenges** technique (**CTF** ou **Capture The Flag**) liée à l'apprentissage de l'environnement **Linux** et aux fondamentaux de la **Cybersécurité** . (overthewire.org)

🚀 **Principe** : Les **challenges** se basent sur un principe **progressif**, où chaque **niveau** impose de résoudre une **problématique spécifique** :

- 🔍 Recherche et manipulation de fichiers
- 📄 Analyse des données
- 📶 Protocoles réseaux

🔒 Afin d'en extraire le **mot de passe** nous permettant d'accéder au niveau suivant (Dans le niveau **bandit1** je trouverai le mot de passe pour accéder à **bandit2**).

Objectif 🎯 : Ce rapport documente ma progression sur les différents niveaux et mes compétences acquise durant les 12 premiers paliers.

Niveau 0 → Niveau 1 :

Consigne :

« The goal of this level is for you to log into the game using SSH. The host to which you need to connect is `bandit.labs.overthewire.org`, on port 2220. The username is `bandit0` and the password is `bandit0`. Once logged in, go to the level 1 page to find out how to beat Level 1.»

« The password for the next level is stored in a file called `readme` located in the home directory. Use this password to log into `bandit1` using SSH. Whenever you find a password for a level, use SSH (on port 2220) to log into that level and continue the game. »

Pour ce premier niveau d'initialisation il nous est simplement demandé de se connecter en SSH au serveur **bandit.labs.overthewire.org** sous l'utilisateur **bandit0** avec le mot de passe **bandit0** via le port **2220**.

La commande étant celle-ci :

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
```

Puis, j'entre le mot de passe « `bandit0` »

Une fois connecté je fais la commande `ls` je m'aperçois de la présence d'un fichier « `readme` », je l'ouvre donc grâce à la commande `cat` `readme` qui m'affiche le mot de passe contenu à l'intérieur.

```
bandit0@bandit:~$ ls
readme
bandit0@bandit:~$ cat readme
Congratulations on your first steps into the bandit game!!
Please make sure you have read the rules at https://overthewire.org/rules/
If you are following a course, workshop, walkthrough or other educational activity,
please inform the instructor about the rules as well and encourage them to
contribute to the OverTheWire community so we can keep these games free!

The password you are looking for is: ZjLjTmM6FvvyRnrb2rfNWOZ0Ta6ip5If
bandit0@bandit:~$
```

Niveau 1 → Niveau 2 :

Consigne :

🎯 | *The password for the next level is stored in a file called - located in the home directory |*

Il m'est demandé de lire un fichier appelé - que nous pouvons appeler « dashed filename »

Le problème c'est qu'un fichier nommé de la sorte ne peut pas être simplement lu via une commande cat ou nano ordinaire, faisons le test :

```
bandit1@bandit:~$ ls
-
bandit1@bandit:~$ cat -

^C
bandit1@bandit:~$ nano -
Reading data from keyboard; type ^D or ^D^D to finish.
Unable to create directory /home/bandit1/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.
```

Sur linux, de nombreuses commandes utilisent le symbole - pour indiquer une option ou autres entrées.

La solution est alors d'utiliser le chemin relatif du fichier pour le lire en ajoutant ./ devant le nom du fichier.

Ce qui donne : `cat ./-`

```
bandit1@bandit:~$ cat ./-
263JGJPfgU6LtdEvgfWU1XP5yac29mFx
```

Alternative :

`cat < -` (le symbole < indiquant que le contenu de - doit être renvoyé vers la commande cat)

```
bandit1@bandit:~$ cat < -
263JGJPfgU6LtdEvgfWU1XP5yac29mFx
```

Niveau 2 → Niveau 3 :

Consigne :

🎯 | The password for the next level is stored in a file called **--spaces in this filename--** located in the home directory |

Il m'est indiqué que le mot de passe se trouve dans un fichier appelé **--spaces in this filename--**, comme dit, ce fichier contient des espaces dans son nom, le problème, c'est qu'on ne peut pas lire simplement un fichier contenant des espaces :

```
bandit2@bandit:~$ cat --spaces in this filename--
cat: unrecognized option '--spaces'
Try 'cat --help' for more information.
bandit2@bandit:~$ nano --spaces in this filename--
nano: unrecognized option '--spaces'
Type 'nano -h' for a list of available options.
```

La commande cat ou nano pense que je tente d'entrer une option, un peu à la manière du premier niveau.

Pour contrer cela il faut ajouter **** avant chaque espace et comme au niveau précédent utiliser **<** ou **./** à cause du double tiret.

Ce qui donne :

```
bandit2@bandit:~$ cat < --spaces\ in\ this\ filename--
MNk8KNH3Usiio41PRUEoDFPqfxLP\Smx
```

Niveau 3 → Niveau 4 :

Consigne :

 | The password for the next level is stored in a hidden file in the **inhere** directory.

La consigne m'indique que le mot de passe est contenu dans un fichier se trouvant dans le dossier **inhere**, je me déplace alors dans ce fichier :

```
bandit3@bandit:~$ cd inhere/  
bandit3@bandit:~/inhere$ ls  
bandit3@bandit:~/inhere$ |
```

Mais il n'y a aucun fichier apparent quand j'utilise simplement la commande `ls`, dans ce cas-là, je peux tenter la commande `ls -a`(all) pour montrer tous les objets présents et même ceux qui sont masqués :

```
bandit3@bandit:~/inhere$ ls -a  
.  ..  ...Hiding-From-You  
bandit3@bandit:~/inhere$
```

On y découvre alors un fichier nommé « **...Hiding-From-You** », les fichiers dont le nom commence par un `.` sont considérés comme cachés, ce qui explique que nous ne le voyions pas.

Je peux alors effectuer ma commande `cat ...Hiding-From-You` :

```
bandit3@bandit:~/inhere$ cat ...Hiding-From-You  
2WmrDFRmJIq3IPxneAaMGhap0pFhF3NJ
```

Niveau 4 → Niveau 5 :

Consigne :

 | The password for the next level is stored in the only human-readable file in the inhere directory.

La consigne indique que le mot de passe se trouve dans le seul fichier lisible par un humain, on peut donc en déduire qu'il y a plusieurs fichiers illisibles et qu'un parmi cette liste est lisible.

```
bandit4@bandit:~$ ls
inhere
bandit4@bandit:~$ cd inhere/
bandit4@bandit:~/inhere$ ls
-file00 -file01 -file02 -file03 -file04 -file05 -file06 -file07 -file08 -file09
bandit4@bandit:~/inhere$
```

Effectivement il y a plusieurs fichiers qui semblent illisible ou corrompu qui en altèrent même le Shell :

```
bandit4@bandit:~/inhere$ cat < -file00
6);/:`d0Jhpfr00}0k'000vjbandit4@bandit:~/inhere$ cat < -file01
7-(c\0;d0030_r00:#0000b02bandit4@bandit:~/inhere$ |
```

Pour ne pas s'embêter à lire les fichiers un par un afin de trouver le bon, nous pouvons utiliser la commande `file` qui nous indique de quel type de fichier il s'agit réellement (sans parler d'extension), en ajoutant l'argument `./*` pour analyser tous les fichiers du dossier.

Ce qui donne `file ./*` :

```
bandit4@bandit:~/inhere$ file ./*
./-file00: data
./-file01: data
./-file02: data
./-file03: data
./-file04: data
./-file05: data
./-file06: data
./-file07: ASCII text
./-file08: data
./-file09: data
bandit4@bandit:~/inhere$ cat ./-file07
4oQYVPkxZOOE005pTW81FB8j8lxXGUQw
```

Nous pouvons voir ici que le fichier **-file07** est le seul fichier identifié comme « **ASCII text** » et le seul lisible :

Niveau 5 → Niveau 6 :

Consigne :

🔗 | The password for the next level is stored in a file somewhere under the `inhere` directory and has all of the following properties:

- `human-readable`
- `1033 bytes in size`
- `not executable`

Dans ce niveau, le fichier est caché dans une arborescence de plusieurs sous-dossiers. Manuellement, il est impossible de le retrouver.

```
bandit5@bandit:~/inhere$ ls
maybeh ere00  maybeh ere02  maybeh ere04  maybeh ere06  maybeh ere08  maybeh ere10  maybeh ere12  maybeh ere14  maybeh ere16  maybeh ere18
maybeh ere01  maybeh ere03  maybeh ere05  maybeh ere07  maybeh ere09  maybeh ere11  maybeh ere13  maybeh ere15  maybeh ere17  maybeh ere19
```

La consigne nous donne quelques éléments pour le retrouver : lisible par un humain, son poids est de 1033 octets et il n'est pas exécutable.

Dans cette situation, la commande `find` nous sera utile en y ajoutant ces arguments :

- `.` : Chercher dans le répertoire courant et ses sous-dossiers.
- `-type f` : Chercher uniquement des fichiers (pas de dossiers).
- `-size 1033c` : Filtrer précisément sur la taille de 1033 octets (c pour caractères/octets).
- `! -executable` : Le point d'exclamation signifie « NOT » (exclure les fichiers exécutables).

Ce qui donne `find . -type f -size 1033c ! -executable` :

```
bandit5@bandit:~/inhere$ find . -type f -size 1033c ! -executable
./maybeh ere07/.file2
bandit5@bandit:~/inhere$ cat ./maybeh ere07/.file2
HWasnPhtq9AVKe0dmk45nxy20cvUa6EG
```

Niveau 6 → Niveau 7 :

Consigne :

🔗 | The password for the next level is stored **somewhere on the server** and has all of the following properties:

- owned by user bandit7
- owned by group bandit6
- 33 bytes in size

A présent, le fichier n'est plus stocké dans notre répertoire personnel mais il peut être n'importe où sur le serveur, nous avons alors quelques indications pour nous aider à le retrouver : le fichier appartient à l'utilisateur bandit7 et au groupe bandit6 et sa taille est de 33 octets.

Comme au précédent niveau, la commande **find** va pouvoir nous aider :

Adaptons les options et arguments aux circonstances données :

- **/** : Chercher directement à la racine du système :
- **-user bandit7 -group bandit6** : Fichier appartenant à l'utilisateur **bandit7** et au group **bandit6**
- **-size 33c** : D'une taille de 33 bytes
- **2>/dev/null** : Rediriger les erreurs (**flux 2**) vers «le vide » (**null**) pour ne voir apparaitre que le fichier auquel nous avons accès et qui nous intéresse.

Ce qui donne **find / -user bandit7 -group bandit6 -size 33c 2>/dev/null** :

```
bandit6@bandit:/$ find / -user bandit7 -group bandit6 -size 33c 2>/dev/null  
/var/lib/dpkg/info/bandit7.password
```

```
bandit6@bandit:~$ cat /var/lib/dpkg/info/bandit7.password  
morbNTDkSW6jIlUc0ymOdMaLnOlFVAaj
```

Niveau 7 → Niveau 8 :

Consigne :

 | *The password for the next level is stored in the file **data.txt** next to the word **millionth***

La consigne nous indique que le mot de passe est stocké à côté du mot **millionth** dans le fichier **data.txt**. Pour cela, la commande **grep** va nous être utile avec cette syntaxe :

```
> grep millionth data.txt
```

La commande **grep** va chercher une ligne dans le fichier **data.txt** qui contient le mot **millionth** :

```
bandit7@bandit:~$ grep millionth data.txt  
millionth      dfwvzFQi4mU0wfNbF0e9RoWskMLg7eEc
```

Niveau 8 → Niveau 9 :

Consigne :

 | *The password for the next level is stored in the file **data.txt** and is the only line of text that occurs only once*

La consigne nous indique que le mot de passe est stocké dans le fichier **data.txt** et qu'il est la seule chaîne de caractère unique dans le fichier, ce qui veut dire que le fichier **data.txt** est probablement rempli de doublons et que le mot de passe est le seul à n'apparaître qu'une seule fois.

Nous allons alors utiliser deux commandes en même temps pour cela, **sort**, une commande qui sert dans un premier temps à trier par ordre alphabétique les lignes du fichier. Puis nous allons utiliser la commande **uniq -u** qui nous permet de n'afficher que la ligne qui n'apparaît qu'une seule fois :

```
bandit8@bandit:~$ sort data.txt | uniq -u
4CKMh1JI91bUIZZPXDqGana14xvAg0JM
bandit8@bandit:~$
```

Niveau 9 → Niveau 10 :

Consigne :

 | The password for the next level is stored in the file **data.txt** in one of the few human-readable strings, preceded by several '=' characters.

On nous dit que le mot de passe est stocké dans le fichier data.txt dans l'une des rares chaînes de caractère lisibles par l'homme, pour retrouver cette chaîne de caractère on nous indique qu'elle est précédée d'une répétition de plusieurs « = »

Nous allons donc utiliser **strings** pour pouvoir lire le fichier qui n'est pas lisible simplement, et dans la même commande utiliser **grep** pour chercher une ligne contenant plusieurs « = »

Ce qui donne :

```
> strings data.txt | grep ==
```

```
bandit9@bandit:~$ strings data.txt | grep ==
===== the
===== password
E===== is
5===== FGUW5ilLVJrxX9kMYMmlN4MgbpfMiqey
```

Niveau 10 → Niveau 11 :

Consigne :

 | The password for the next level is stored in the file **data.txt**, which contains base64 encoded data

Le mot de passe est stocké dans le fichier **data.txt** codé en base64 .

Il nous suffit de le lire grâce à la commande **base64 -d** (decode)

Ce qui donne :

```
> base64 -d
```

```
bandit10@bandit:~$ base64 -d data.txt  
The password is dtR173fZKb0RRsDFSGsg2RWnpNVj3qRr
```

Niveau 11 → Niveau 12 :

Consigne :

🔗 | The password for the next level is stored in the file **data.txt**, where all lowercase (a-z) and uppercase (A-Z) letters have been rotated by 13 positions

Le fichier contient du texte qui semble lisible (caractères ASCII) mais qui ne veut rien dire.

```
bandit11@bandit:~$ cat data.txt
Gur cnffjbeq vf 7k16JArUVv5LxVuJfsSVdbbtaHG1w9D4
```

Il s'agit en effet d'un chiffrement ROT13 (rotation de 13 caractères), étant donné que l'alphabet est composé de 26 lettres il suffit de faire une rotation de 13 de plus pour revenir aux caractères d'origine en utilisant cette commande :

```
> cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Explication :

- **A-Za-z** : La commande doit agir sur tout l'alphabet (de A à Z) en majuscule et en minuscule
- **N-ZA-M** : On indique que l'on veut décaler les lettres de **N** à **Z** dans le périmètre de **A** à **M**.
- **n-za-m** : On effectue le même décalage pour les minuscules

Essayons la commande :

```
bandit11@bandit:~$ cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
The password is 7x16WNeHIi5YkIhWsfFIqoognUTyj9Q4
```

Niveau 12 → Niveau 13 :

Consigne :

🔗 | The password for the next level is stored in the file **data.txt**, which is a hexdump of a file that has been repeatedly compressed. For this level it may be useful to create a directory under /tmp in which you can work. Use mkdir with a hard to guess directory name. Or better, use the command "mktemp -d". Then copy the datafile using cp, and rename it using mv

Dans ce niveau nous devons faire preuve de patience et de méthodologie, le fichier d'origine est un « **hexdump** » (représentation en texte de données binaires) qui cache une succession de compressions dans différents formats (Gzip, Bzip2, Tar). Nous pouvons voir cela un peu comme une poupée Russe, un fichier compressé qui contient un autre fichier compressé qui à son tour en contient un autre et ainsi de suite. Il nous est aussi indiqué que dans le répertoire personnel nous n'avons pas les droits pour créer des fichiers , alors il faut alors créer un dossier dans le **/tmp** avec la commande **mktemp -d**

```
bandit12@bandit:~$ mktemp -d
/tmp/tmp.y9p1AJTqks
```

Premièrement, nous devons **inverser** (ou décoder) le **hexdump** grâce à la commande **xxd -r** (revert) en y ajoutant **> /tmp/tmp.y9p1AJTqks/data.bin** pour rediriger le résultat de la commande dans un fichier afin de le stocker dans le dossier temporaire que nous venons de créer :

```
> xxd -r data.txt > /tmp/tmp.y9p1AJTqks/data.bin
```

```
bandit12@bandit:~$ xxd -r data.txt > /tmp/tmp.y9p1AJTqks/data.bin
bandit12@bandit:~$ cd /tmp/
bandit12@bandit:/tmp$ cd tmp.y9p1AJTqks
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data.bin
```

Une fois notre hexdump inversé, on peut effectuer la commande **file** pour savoir de quel type de fichier il s'agit afin de pouvoir le décompresser par la suite .

```
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data.bin
data.bin: gzip compressed data, was "data2.bin", last
```

Nous pouvons voir qu'il s'agit d'un fichier compressé avec **gzip** nommé auparavant **data2.bin** , pour effectuer la décompression de celui-ci nous devons

d'abord renommer le fichier avec la bonne **extension**, donc pour **gzip** il s'agit de **.gz** -> **mv data.bin data.gz**

Puis l'extraire avec la commande **gunzip** :

```
> gunzip data.gz
```

Ainsi de suite pour les prochains fichiers et format en utilisant les bonnes commandes en fonction de ceux si :

```
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data.bin
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data.bin
data.bin: gzip compressed data, was "data2.bin", last modified:
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ mv data.bin data.gz
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data.gz
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ gunzip data.gz
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data
data: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ mv data data.bz2
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data.bz2
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ bunzip2 data.bz2
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data
data: gzip compressed data, was "data4.bin", last modified:
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ mv data data.gz
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ gunzip data.gz
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ mv data data.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ tar -xf data.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data5.bin  data.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data5.bin
data5.bin: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ mv data5.bin data5.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ tar -xf data5.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data5.tar  data6.bin  data.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ mv data6.bin data6.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ tar -xf data6.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data5.tar  data6.tar  data8.bin  data.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data8.bin
data8.bin: gzip compressed data, was "data9.bin", last modified:
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ mv data8.bin data8.gz
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ gunzip data8.gz
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ ls
data5.tar  data6.tar  data8  data.tar
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ file data8
data8: ASCII text
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ cat data8
The password is F05dwFsc0cbaIiH0h8J2eUks2vdTDwAn
bandit12@bandit:/tmp/tmp.y9p1AJTqks$ |
```

Conclusion :

La validation de ces 12 premiers niveaux, axés sur la **manipulation de fichiers** et l'**administration système**, a été une étape déterminante. Ces bases fondamentales m'ont permis d'acquérir une réelle fluidité dans l'environnement Linux, une aisance indispensable pour aborder sereinement les challenges plus complexes à venir.